

# Database Design Document

**Enda Eyesus Community Digital Ecosystem & LMS**

6/1/2026

Koinonia Hub

Kibrom Abebe

## Contents

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>1. Introduction .....</b>                                  | <b>3</b>  |
| <b>1.1 Purpose .....</b>                                      | <b>4</b>  |
| <b>1.2 Scope.....</b>                                         | <b>4</b>  |
| <b>1.3 Database Platform.....</b>                             | <b>4</b>  |
| <b>2. Database Environment &amp; Standards .....</b>          | <b>4</b>  |
| <b>2.1 Naming Conventions .....</b>                           | <b>4</b>  |
| <b>2.2 Data Types .....</b>                                   | <b>5</b>  |
| <b>2.3 Audit Fields.....</b>                                  | <b>5</b>  |
| <b>3. Conceptual &amp; Logical Data Model .....</b>           | <b>5</b>  |
| <b>3.1 Entity-Relationship Diagram (ERD) – Mermaid.....</b>   | <b>5</b>  |
| <b>3.2 Entity Descriptions .....</b>                          | <b>5</b>  |
| <b>4. Physical Schema Design .....</b>                        | <b>6</b>  |
| <b>4.1 Enums (Custom Types) .....</b>                         | <b>6</b>  |
| <b>4.2 Table DDL (Core Tables).....</b>                       | <b>6</b>  |
| <b>5. Indexes &amp; Performance Optimization .....</b>        | <b>11</b> |
| <b>5.1 Index Strategy .....</b>                               | <b>11</b> |
| <b>5.2 Partial &amp; Expression Indexes .....</b>             | <b>11</b> |
| <b>5.3 Recommended Maintenance .....</b>                      | <b>12</b> |
| <b>6. Constraints &amp; Data Integrity .....</b>              | <b>12</b> |
| <b>6.1 Primary Keys .....</b>                                 | <b>12</b> |
| <b>6.2 Foreign Keys.....</b>                                  | <b>12</b> |
| <b>6.3 Check Constraints .....</b>                            | <b>13</b> |
| <b>6.4 Business Rule Triggers.....</b>                        | <b>13</b> |
| <b>7. Partitioning Strategy.....</b>                          | <b>14</b> |
| <b>7.1 Tables Suitable for Partitioning .....</b>             | <b>14</b> |
| <b>7.2 Partitioning Scheme for audit_logs (Example) .....</b> | <b>14</b> |
| <b>7.3 Archival &amp; Cleanup.....</b>                        | <b>15</b> |
| <b>8. Security &amp; Access Control.....</b>                  | <b>15</b> |

|                                                                                   |    |
|-----------------------------------------------------------------------------------|----|
| 8.1 Database Roles .....                                                          | 15 |
| 8.2 Row-Level Security (RLS) .....                                                | 16 |
| 8.3 Audit Logging (Immutable) .....                                               | 16 |
| 9. Migration Scripts.....                                                         | 16 |
| 9.1 Migration File Naming Convention.....                                         | 17 |
| 9.2 Base Schema Migration (Initial) .....                                         | 17 |
| 9.3 Sample Migration: Add phone_number Column .....                               | 17 |
| 9.4 Data Seeding (Initial Data) .....                                             | 18 |
| 10. Backup & Recovery .....                                                       | 18 |
| 10.1 Backup Strategy.....                                                         | 18 |
| 10.2 Recovery Testing .....                                                       | 18 |
| 10.3 Point-in-Time Recovery (PITR) Example .....                                  | 18 |
| 11. Appendices.....                                                               | 19 |
| Appendix A – Entity Relationship Diagram (Text Only).....                         | 19 |
| Appendix B – PostgreSQL Configuration Recommendations (RDS Parameter Group) ..... | 19 |
| Appendix C – Sample Query Patterns with Index Usage .....                         | 20 |

# 1. Introduction

## 1.1 Purpose

This Database Design Document (DBD) defines the structure, relationships, constraints, indexing strategy, partitioning plan, and migration scripts for the PostgreSQL database powering the Enda Eyesus Community Digital Ecosystem and LMS. It is derived from the SRS v2.0 and System Architecture Design, and serves as the authoritative reference for database administrators and developers.

## 1.2 Scope

The document covers:

- Entity-Relationship Diagram (ERD) in both textual and graphical (Mermaid) form
- Complete DDL (Data Definition Language) for all tables, sequences, enums
- Primary keys, foreign keys, check constraints, unique constraints
- Indexing strategy for query performance
- Partitioning approach for audit logs and large tables
- Security (row-level security, roles)
- Migration scripts (versioned, reversible)

## 1.3 Database Platform

| Property      | Value                              |
|---------------|------------------------------------|
| DBMS          | PostgreSQL 15+                     |
| Hosting       | AWS RDS (Multi-AZ)                 |
| Character set | UTF8                               |
| Timezone      | UTC (all timestamps stored in UTC) |
| Extensions    | uuid-oss, pgcrypto, btree_gin      |

# 2. Database Environment & Standards

## 2.1 Naming Conventions

| Object             | Convention             | Example                                 |
|--------------------|------------------------|-----------------------------------------|
| Tables             | snake_case, plural     | users, service_classes, lms_enrollments |
| Primary key        | id (always)            | id                                      |
| Foreign key        | {referenced_table}_id  | service_class_id                        |
| Indexes            | idx_{table}_{columns}  | idx_users_email                         |
| Unique constraints | uq_{table}_{columns}   | uq_users_email                          |
| Check constraints  | ck_{table}_{condition} | ck_users_academic_year                  |

|                  |                      |                                 |
|------------------|----------------------|---------------------------------|
| <b>Sequences</b> | {table}_seq          | batch_number_seq                |
| <b>Enums</b>     | snake_case, singular | course_track, submission_status |

## 2.2 Data Types

| Use Case                             | PostgreSQL Type                     |
|--------------------------------------|-------------------------------------|
| <b>Identifiers</b>                   | UUID (primary keys)                 |
| <b>Short strings (&lt;100 chars)</b> | VARCHAR(n)                          |
| <b>Long text</b>                     | TEXT                                |
| <b>Boolean flags</b>                 | BOOLEAN                             |
| <b>Timestamps</b>                    | TIMESTAMPTZ (always with time zone) |
| <b>Numeric grades</b>                | NUMERIC(5,2) (0.00 – 100.00)        |
| <b>JSON structures</b>               | JSONB                               |
| <b>Enum (fixed set)</b>              | ENUM (custom type)                  |
| <b>Counters / years</b>              | INTEGER                             |

## 2.3 Audit Fields

Every table that stores user-generated or mutable data includes:

| Column     | Type            | Purpose                                                |
|------------|-----------------|--------------------------------------------------------|
| created_at | TIMESTAMPTZ     | Record creation timestamp (default CURRENT_TIMESTAMP)  |
| updated_at | TIMESTAMPTZ     | Last modification timestamp (auto-updated via trigger) |
| created_by | UUID (nullable) | User who created the record (references users.id)      |
| updated_by | UUID (nullable) | User who last modified the record                      |

For audit/log tables (immutable), only created\_at and created\_by are present.

## 3. Conceptual & Logical Data Model

### 3.1 Entity-Relationship Diagram (ERD) – Mermaid

### 3.2 Entity Descriptions

| Entity          | Description                                                                                |
|-----------------|--------------------------------------------------------------------------------------------|
| users           | All system users (Tier1 USER, MEMBER, TEACHER, SERVICE_MANAGER, Secretariat, Global Admin) |
| service_classes | Top-level departments (e.g., Education, Choir, Member Affairs) plus “Default Class”        |
| sub_classes     | Sub-units under a service class (e.g., “Sunday Sub-class”)                                 |
| lms_batches     | Academic cohorts (HC) for each course track                                                |

|                    |                                                                         |
|--------------------|-------------------------------------------------------------------------|
| lms_enrollments    | Junction table linking users to batches with progress data              |
| course_submissions | Teacher content submissions; tracks manual workflow (Draft → Published) |
| announcements      | Public or department-scoped announcements                               |
| comments           | Threaded comments on announcements (max 1-level nesting)                |
| reactions          | Likes / Stars on announcements                                          |
| library_items      | Resource Library entries (Google Drive URLs)                            |
| notifications      | In-app notifications for users                                          |
| audit_logs         | Immutable audit trail for state-changing actions                        |

## 4. Physical Schema Design

### 4.1 Enums (Custom Types)

```
sql
-- Course tracks
CREATE TYPE course_track AS ENUM ('GUBAE_ABEW', 'GUBAE_HAWARYAT', 'GUBAE_ECCLESIAE');

-- Batch status
CREATE TYPE batch_status AS ENUM ('ACTIVE', 'GRADUATED');

-- System roles
CREATE TYPE system_role AS ENUM (
    'USER', 'MEMBER', 'TEACHER', 'SERVICE_MANAGER',
    'SECRETARIAT_SECRETARY', 'SECRETARIAT_VICE', 'SECRETARIAT_CHAIRMAN'
);

-- Submission status (manual workflow)
CREATE TYPE submission_status AS ENUM (
    'DRAFT', 'SUBMITTED', 'UNDER_REVIEW', 'APPROVED',
    'REJECTED', 'IMPLEMENTATION_IN_PROGRESS', 'PUBLISHED'
);

-- Reaction type
CREATE TYPE reaction_type AS ENUM ('LIKE', 'STAR');

-- Library category
CREATE TYPE library_category AS ENUM ('SPIRITUAL', 'ACADEMIC', 'OTHER');

-- Document type (academic)
CREATE TYPE document_type AS ENUM ('TEXTBOOK', 'PAST_EXAM');
```

## 4.2 Table DDL (Core Tables)

### 4.2.1 users

```
sql
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  full_name_three_parts VARCHAR(150) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  university_id VARCHAR(50) UNIQUE,
  academic_dept VARCHAR(100),
  academic_year INTEGER CHECK (academic_year BETWEEN 1 AND 5),
  dorm_block VARCHAR(20),
  dorm_room VARCHAR(20),
  phone_number VARCHAR(20),
  profile_image_url VARCHAR(255) DEFAULT '/assets/avatar.png',
  bio TEXT,
  system_role system_role NOT NULL DEFAULT 'USER',
  service_class_id UUID REFERENCES service_classes(id) ON DELETE SET NULL,
  repentance_father_id UUID REFERENCES users(id) ON DELETE SET NULL,
  repentance_deacon_id UUID REFERENCES users(id) ON DELETE SET NULL,
  spiritual_father_id UUID REFERENCES users(id) ON DELETE SET NULL,
  spiritual_mother_id UUID REFERENCES users(id) ON DELETE SET NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  created_by UUID REFERENCES users(id) ON DELETE SET NULL,
  updated_by UUID REFERENCES users(id) ON DELETE SET NULL
);

-- Auto-update updated_at trigger
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = CURRENT_TIMESTAMP;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_users_updated_at
BEFORE UPDATE ON users
FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
```

### 4.2.2 service\_classes

```
sql
CREATE TABLE service_classes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

class_name_amharic VARCHAR(100) NOT NULL UNIQUE,
is_public_registration BOOLEAN NOT NULL DEFAULT TRUE,
created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

```

### 4.2.3 sub\_classes

```

sql
CREATE TABLE sub_classes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  parent_class_id UUID NOT NULL REFERENCES service_classes(id) ON DELETE CASCADE,
  sub_class_name VARCHAR(100) NOT NULL,
  sub_chair_id UUID REFERENCES users(id) ON DELETE SET NULL,
  sub_vice_id UUID REFERENCES users(id) ON DELETE SET NULL,
  sub_secretary_id UUID REFERENCES users(id) ON DELETE SET NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

```

### 4.2.4 lms\_batches

```

sql
CREATE TABLE lms_batches (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  course_track course_track NOT NULL,
  batch_number INTEGER NOT NULL,
  status batch_status NOT NULL DEFAULT 'ACTIVE',
  academic_year INTEGER NOT NULL CHECK (academic_year BETWEEN 2000 AND 2100),
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Ensure only one ACTIVE batch per course track
CREATE UNIQUE INDEX uq_lms_batches_active_track
  ON lms_batches (course_track)
  WHERE status = 'ACTIVE';

```

### 4.2.5 lms\_enrollments

```

sql
CREATE TABLE lms_enrollments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  batch_id UUID NOT NULL REFERENCES lms_batches(id) ON DELETE CASCADE,
  completed_lessons JSONB NOT NULL DEFAULT '[]',
  quiz_scores JSONB NOT NULL DEFAULT '{}',
  final_exam_score NUMERIC(5,2) CHECK (final_exam_score >= 0 AND final_exam_score <= 100),
  is_passed BOOLEAN NOT NULL DEFAULT FALSE,

```

```

certificate_hash VARCHAR(255) UNIQUE,
enrolled_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
UNIQUE(user_id, batch_id)
);

```

#### 4.2.6 course\_submissions

```

sql
CREATE TABLE course_submissions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  teacher_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  batch_id UUID NOT NULL REFERENCES lms_batches(id) ON DELETE CASCADE,
  title VARCHAR(200) NOT NULL,
  content_package TEXT NOT NULL,
  status submission_status NOT NULL DEFAULT 'DRAFT',
  review_feedback TEXT,
  implemented_page_url VARCHAR(500),
  submitted_at TIMESTAMPTZ,
  reviewed_at TIMESTAMPTZ,
  implemented_at TIMESTAMPTZ,
  published_at TIMESTAMPTZ,
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

```

#### 4.2.7 announcements, comments, reactions

```

sql
CREATE TABLE announcements (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title VARCHAR(200) NOT NULL,
  content TEXT NOT NULL,
  is_public BOOLEAN NOT NULL DEFAULT TRUE,
  target_class_id UUID REFERENCES service_classes(id) ON DELETE CASCADE,
  author_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  published_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE comments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  announcement_id UUID NOT NULL REFERENCES announcements(id) ON DELETE CASCADE,
  parent_comment_id UUID REFERENCES comments(id) ON DELETE CASCADE,
  author_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  content TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

```

```
);

CREATE TABLE reactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  announcement_id UUID NOT NULL REFERENCES announcements(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  reaction_type reaction_type NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(announcement_id, user_id)
);
```

#### 4.2.8 library\_items

```
sql
CREATE TABLE library_items (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title VARCHAR(200) NOT NULL,
  description TEXT,
  drive_url VARCHAR(500) NOT NULL,
  category library_category NOT NULL,
  academic_department VARCHAR(100),
  academic_year INTEGER CHECK (academic_year BETWEEN 1 AND 5),
  course_id VARCHAR(50),
  document_type document_type,
  likes INTEGER NOT NULL DEFAULT 0,
  downloads INTEGER NOT NULL DEFAULT 0,
  is_link_broken BOOLEAN NOT NULL DEFAULT FALSE,
  last_checked_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

#### 4.2.9 notifications

```
sql
CREATE TABLE notifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  title VARCHAR(200) NOT NULL,
  message TEXT NOT NULL,
  target_route VARCHAR(200),
  is_read BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

#### 4.2.10 audit\_logs

```
sql
CREATE TABLE audit_logs (
```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID, -- can be NULL for system actions
action VARCHAR(50) NOT NULL,
entity_type VARCHAR(50) NOT NULL,
entity_id UUID,
old_state JSONB,
new_state JSONB,
ip_address INET,
created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

```

## 5. Indexes & Performance Optimization

### 5.1 Index Strategy

| Table              | Index Columns                  | Type             | Purpose                                  |
|--------------------|--------------------------------|------------------|------------------------------------------|
| users              | email                          | UNIQUE<br>B-tree | Fast login lookup                        |
| users              | university_id                  | UNIQUE<br>B-tree | Membership verification                  |
| users              | system_role                    | B-tree           | Filtering by role                        |
| users              | service_class_id               | B-tree           | Department joins                         |
| users              | repentance_father_id           | B-tree           | Pastoral queries                         |
| users              | created_at                     | B-tree           | Reporting by date                        |
| lms_enrollments    | (user_id, batch_id)            | UNIQUE<br>B-tree | Enrollment check                         |
| lms_enrollments    | batch_id                       | B-tree           | Batch roster queries                     |
| lms_enrollments    | is_passed                      | B-tree           | Graduation filtering                     |
| lms_batches        | status (partial index)         | B-tree           | Find active batch: WHERE status='ACTIVE' |
| course_submissions | (batch_id, status)             | B-tree           | Workflow queries                         |
| course_submissions | teacher_id                     | B-tree           | Teacher's submissions                    |
| announcements      | (is_public, published_at)      | B-tree           | Public feed                              |
| announcements      | target_class_id                | B-tree           | Department-scoped announcements          |
| comments           | announcement_id                | B-tree           | Fetch comments for an announcement       |
| reactions          | (announcement_id, user_id)     | UNIQUE<br>B-tree | Prevent duplicate reactions              |
| library_items      | category                       | B-tree           | Category filtering                       |
| library_items      | academic_department            | B-tree           | Academic drill-down                      |
| notifications      | (user_id, is_read, created_at) | B-tree           | User's unread notifications              |

|            |                          |        |                                    |
|------------|--------------------------|--------|------------------------------------|
| audit_logs | (entity_type, entity_id) | B-tree | Retrieve audit trail for an entity |
|------------|--------------------------|--------|------------------------------------|

## 5.2 Partial & Expression Indexes

```

sql
-- Find only ACTIVE batches quickly
CREATE INDEX idx_lms_batches_active ON lms_batches (course_track) WHERE status = 'ACTIVE';

-- Find only non-archived users (role != 'ARCHIVED_ALUMNI')
CREATE INDEX idx_users_active ON users (system_role) WHERE system_role != 'ARCHIVED_ALUMNI';

-- Full-text search on announcement titles (requires extension)
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX idx_announcements_title_trgm ON announcements USING GIN (title gin_trgm_ops);

-- JSONB index for quiz scores (if you frequently query specific quiz keys)
CREATE INDEX idx_lms_enrollments_quiz_scores ON lms_enrollments USING GIN (quiz_scores);

```

## 5.3 Recommended Maintenance

- **Weekly:** ANALYZE on high-traffic tables (users, lms\_enrollments, announcements).
- **Monthly:** Reindex if bloat detected (pg\_repack or REINDEX INDEX CONCURRENTLY).
- **Vacuum:** Auto-vacuum enabled; vacuum audit\_logs daily due to high insert rate.

# 6. Constraints & Data Integrity

## 6.1 Primary Keys

All tables use UUID primary keys generated by gen\_random\_uuid() (v4). This avoids sequence collisions in distributed environments.

## 6.2 Foreign Keys

| Table                       | Foreign Key        | On Delete | Rationale                                                                     |
|-----------------------------|--------------------|-----------|-------------------------------------------------------------------------------|
| users.service_class_id      | service_classes.id | SET NULL  | When a department is deleted, members become unassigned (later reassignable). |
| users.repentance_father_id  | users.id           | SET NULL  | Mentor deletion should not delete student.                                    |
| sub_classes.parent_class_id | service_classes.id | CASCADE   | Deleting a service class removes its sub-classes.                             |
| sub_classes.sub_chair_id    | users.id           | SET NULL  | User leaves sub-class leadership; keep the sub-class.                         |

|                               |                  |          |                                                                |
|-------------------------------|------------------|----------|----------------------------------------------------------------|
| lms_enrollments.user_id       | users.id         | CASCADE  | GDPR-style removal: user account deletion removes enrollments. |
| lms_enrollments.batch_id      | lms_batches.id   | CASCADE  | If batch is deleted (rare), enrollments go with it.            |
| course_submissions.teacher_id | users.id         | CASCADE  | Teacher account deletion removes submissions.                  |
| announcements.author_id       | users.id         | CASCADE  | User deletion removes their announcements.                     |
| comments.announcement_id      | announcements.id | CASCADE  | Announcement deletion removes its comments.                    |
| reactions.announcement_id     | announcements.id | CASCADE  | Announcement deletion removes reactions.                       |
| notifications.user_id         | users.id         | CASCADE  | User deletion removes notifications.                           |
| audit_logs.user_id            | users.id         | SET NULL | Keep audit log even if user deleted.                           |

## 6.3 Check Constraints

```
sql
-- Academic year range (1-5)
ALTER TABLE users ADD CONSTRAINT ck_users_academic_year
    CHECK (academic_year IS NULL OR (academic_year BETWEEN 1 AND 5));

-- Final exam score range
ALTER TABLE lms_enrollments ADD CONSTRAINT ck_lms_enrollments_score
    CHECK (final_exam_score IS NULL OR (final_exam_score BETWEEN 0 AND 100));

-- Library item academic year range
ALTER TABLE library_items ADD CONSTRAINT ck_library_academic_year
    CHECK (academic_year IS NULL OR (academic_year BETWEEN 1 AND 5));

-- Course submission title length
ALTER TABLE course_submissions ADD CONSTRAINT ck_submissions_title
    CHECK (char_length(title) >= 3);
```

## 6.4 Business Rule Triggers

### Trigger: Prevent enrollment in advanced course without prerequisite

```
sql
CREATE OR REPLACE FUNCTION check_course_prerequisite()
RETURNS TRIGGER AS $$
DECLARE
    prev_course track;
    has_passed BOOLEAN;
BEGIN
    -- Get the course track of the batch
    SELECT course_track INTO prev_course FROM lms_batches WHERE id = NEW.batch_id;
```

```

-- Determine prerequisite based on target course
IF prev_course = 'GUBAE_HAWARYAT' THEN
    -- Must have passed GUBAE_ABEW
    SELECT EXISTS (
        SELECT 1 FROM lms_enrollments e
        JOIN lms_batches b ON e.batch_id = b.id
        WHERE e.user_id = NEW.user_id AND b.course_track = 'GUBAE_ABEW' AND e.is_passed = TRUE
    ) INTO has_passed;
    IF NOT has_passed THEN
        RAISE EXCEPTION 'Prerequisite not met: Gubae Abew is required before Hawaryat';
    END IF;
ELSIF prev_course = 'GUBAE_ECCLESIAE' THEN
    -- Must have passed GUBAE_HAWARYAT
    SELECT EXISTS (
        SELECT 1 FROM lms_enrollments e
        JOIN lms_batches b ON e.batch_id = b.id
        WHERE e.user_id = NEW.user_id AND b.course_track = 'GUBAE_HAWARYAT' AND e.is_passed = TRUE
    ) INTO has_passed;
    IF NOT has_passed THEN
        RAISE EXCEPTION 'Prerequisite not met: Gubae Hawaryat is required before Ecclesiae';
    END IF;
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_prerequisite
BEFORE INSERT OR UPDATE ON lms_enrollments
FOR EACH ROW
EXECUTE FUNCTION check_course_prerequisite();

```

## 7. Partitioning Strategy

### 7.1 Tables Suitable for Partitioning

| Table         | Estimated Growth           | Partition Key          | Retention                     |
|---------------|----------------------------|------------------------|-------------------------------|
| audit_logs    | High (every state change)  | created_at (monthly)   | 2 years (archive older)       |
| notifications | Medium (every user action) | created_at (monthly)   | 6 months (keep unread longer) |
| comments      | Medium                     | created_at (quarterly) | 3 years                       |

### 7.2 Partitioning Scheme for audit\_logs (Example)

We use **declarative partitioning** (PostgreSQL 10+).

```

sql
-- Create parent table
CREATE TABLE audit_logs (
    id UUID DEFAULT gen_random_uuid(),
    user_id UUID,
    action VARCHAR(50) NOT NULL,
    entity_type VARCHAR(50) NOT NULL,
    entity_id UUID,
    old_state JSONB,
    new_state JSONB,
    ip_address INET,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
) PARTITION BY RANGE (created_at);

-- Create monthly partitions (example for 2025)
CREATE TABLE audit_logs_2025_01 PARTITION OF audit_logs
    FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');

CREATE TABLE audit_logs_2025_02 PARTITION OF audit_logs
    FOR VALUES FROM ('2025-02-01') TO ('2025-03-01');

-- ... continue for each month

-- Index on each partition (automatically inherited)
CREATE INDEX idx_audit_logs_created_at ON audit_logs (created_at);

```

### Automated partition creation (using pg\_partman extension):

```

sql
CREATE EXTENSION IF NOT EXISTS pg_partman;

SELECT partman.create_parent(
    p_parent_table => 'public.audit_logs',
    p_control => 'created_at',
    p_type => 'native',
    p_interval => 'monthly',
    p_premake => 3    -- create partitions 3 months ahead
);

```

## 7.3 Archival & Cleanup

- Old partitions (older than 2 years for audit\_logs) can be **detached** and moved to a cold storage table or exported to CSV and deleted.
- For notifications, after 6 months, soft-delete (mark as archived) and then physically delete after another 6 months using a cron job.

## 8. Security & Access Control

### 8.1 Database Roles

| Role          | Purpose                            | Granted Privileges                                                           |
|---------------|------------------------------------|------------------------------------------------------------------------------|
| app_user      | Application backend (NestJS)       | SELECT, INSERT, UPDATE, DELETE on all tables except audit_logs (insert only) |
| app_reader    | Read-only reports (e.g., Metabase) | SELECT on all tables                                                         |
| app_migration | Running migrations                 | CREATE, ALTER, DROP (only during deployment)                                 |
| app_admin     | Manual DBA tasks                   | ALL PRIVILEGES (restricted to specific IPs)                                  |

### 8.2 Row-Level Security (RLS)

Enable RLS for users table so that a user can only see their own profile unless they have administrative roles.

```
sql
ALTER TABLE users ENABLE ROW LEVEL SECURITY;

CREATE POLICY users_self_policy ON users
  USING (id = current_setting('app.current_user_id')::UUID)
  WITH CHECK (id = current_setting('app.current_user_id')::UUID);

-- Allow service managers to see members in their department
CREATE POLICY users_department_manager_policy ON users
  USING (
    EXISTS (
      SELECT 1 FROM users manager
      WHERE manager.id = current_setting('app.current_user_id')::UUID
      AND manager.system_role = 'SERVICE_MANAGER'
      AND manager.service_class_id = users.service_class_id
    )
  )
  WITH CHECK (FALSE); -- read only
```

**Note:** The application sets `current_setting('app.current_user_id')` at the start of each request (via NestJS middleware).

### 8.3 Audit Logging (Immutable)

The `audit_logs` table is **append-only**. No UPDATE or DELETE triggers allowed. To enforce:

```
sql
REVOKE UPDATE, DELETE ON audit_logs FROM app_user;
```

## 9. Migration Scripts

We use **versioned migrations** (e.g., with `pg-migrate` or `TypeORM` migrations). Each migration file has an **up** and **down** script.

### 9.1 Migration File Naming Convention

YYYYMMDDHHMMSS\_description.sql

Example: 20250601120000\_create\_initial\_schema.sql

### 9.2 Base Schema Migration (Initial)

```
sql
-- 20250601120000_create_initial_schema.sql

-- UP
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
-- ... (all CREATE TYPE and CREATE TABLE statements from Section 4)
-- ... (indexes, triggers, RLS policies)

-- DOWN
DROP TABLE IF EXISTS audit_logs CASCADE;
DROP TABLE IF EXISTS notifications CASCADE;
DROP TABLE IF EXISTS library_items CASCADE;
DROP TABLE IF EXISTS reactions CASCADE;
DROP TABLE IF EXISTS comments CASCADE;
DROP TABLE IF EXISTS announcements CASCADE;
DROP TABLE IF EXISTS course_submissions CASCADE;
DROP TABLE IF EXISTS lms_enrollments CASCADE;
DROP TABLE IF EXISTS lms_batches CASCADE;
DROP TABLE IF EXISTS sub_classes CASCADE;
DROP TABLE IF EXISTS service_classes CASCADE;
DROP TABLE IF EXISTS users CASCADE;
DROP TYPE IF EXISTS system_role;
DROP TYPE IF EXISTS course_track;
DROP TYPE IF EXISTS batch_status;
DROP TYPE IF EXISTS submission_status;
DROP TYPE IF EXISTS reaction_type;
DROP TYPE IF EXISTS library_category;
DROP TYPE IF EXISTS document_type;
```

### 9.3 Sample Migration: Add phone\_number Column

```
sql
-- 20250601150000_add_phone_number_to_users.sql
```

```
-- UP
ALTER TABLE users ADD COLUMN phone_number VARCHAR(20);
-- optionally populate from existing data (not applicable for new field)

-- DOWN
ALTER TABLE users DROP COLUMN phone_number;
```

9.4 Data Seeding (Initial Data)

Create a separate migration for initial static data:

```
sql
-- 20250601180000_seed_initial_data.sql

-- UP
INSERT INTO service_classes (id, class_name_amharic, is_public_registration) VALUES
    (gen_random_uuid(), 'Default Class', FALSE),
    (gen_random_uuid(), 'ትምህርት ክፍል', TRUE),
    (gen_random_uuid(), 'ማህሌት ክፍል', TRUE),
    (gen_random_uuid(), 'አባላት ጉዳይ', TRUE);

-- DOWN
DELETE FROM service_classes WHERE class_name_amharic IN ('Default Class', 'ትምህርት ክፍል', 'ማህሌት ክፍል',
    'አባላት ጉዳይ');
```

10. Backup & Recovery

10.1 Backup Strategy

| Type                      | Frequency             | Retention                  | Method                            |
|---------------------------|-----------------------|----------------------------|-----------------------------------|
| Automated snapshots (RDS) | Daily                 | 30 days                    | AWS RDS snapshot                  |
| Continuous backups (WAL)  | Every 5 minutes       | 7 days                     | RDS Point-in-Time Recovery (PITR) |
| Manual full backup        | Before major releases | Permanent (minimum 1 year) | pg_dump to S3                     |

10.2 Recovery Testing

- **Monthly:** Restore latest snapshot to a staging environment and verify data integrity.
- **Quarterly:** Perform a full DR failover drill (RTO < 4h, RPO ≤ 24h).

## 10.3 Point-in-Time Recovery (PITR) Example

```
bash
# Restore to a specific timestamp (e.g., 10 minutes before accidental deletion)
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier endaeyesus-prod \
  --target-db-instance-identifier endaeyesus-prod-pitr \
  --restore-time "2025-06-01T11:50:00Z"
```

## 11. Appendices

### Appendix A – Entity Relationship Diagram (Text Only)

| Entity             | Key Fields                            | Foreign Keys                                                                  |
|--------------------|---------------------------------------|-------------------------------------------------------------------------------|
| users              | id, email, system_role                | service_class_id → service_classes.id;<br>repentance_father_id → users.id ... |
| service_classes    | id, class_name_amharic                | (none)                                                                        |
| sub_classes        | id, parent_class_id,<br>sub_chair_id  | parent_class_id → service_classes.id; sub_chair_id → users.id                 |
| lms_batches        | id, course_track, status              | (none)                                                                        |
| lms_enrollments    | user_id, batch_id                     | user_id → users.id; batch_id → lms_batches.id                                 |
| course_submissions | teacher_id, batch_id                  | teacher_id → users.id; batch_id → lms_batches.id                              |
| announcements      | author_id, target_class_id            | author_id → users.id; target_class_id → service_classes.id                    |
| comments           | announcement_id,<br>parent_comment_id | announcement_id → announcements.id;<br>parent_comment_id → comments.id        |
| reactions          | announcement_id, user_id              | announcement_id → announcements.id; user_id → users.id                        |
| library_items      | (none)                                | (none)                                                                        |
| notifications      | user_id                               | user_id → users.id                                                            |
| audit_logs         | user_id                               | user_id → users.id                                                            |

### Appendix B – PostgreSQL Configuration Recommendations (RDS Parameter Group)

| Parameter       | Value               | Reason                                                          |
|-----------------|---------------------|-----------------------------------------------------------------|
| max_connections | 200                 | Adjust based on ECS task count (each task uses ~10 connections) |
| shared_buffers  | 25% of instance RAM | Typical PostgreSQL tuning                                       |

|                                      |                        |                                      |
|--------------------------------------|------------------------|--------------------------------------|
| effective_cache_size                 | 75% of RAM             | Optimises index scan decisions       |
| work_mem                             | 8MB                    | Can be increased for complex queries |
| maintenance_work_mem                 | 512MB                  | For VACUUM, CREATE INDEX             |
| random_page_cost                     | 1.1 (if using SSD)     | Lower cost for indexed scans         |
| log_min_duration_statement           | 500ms                  | Log slow queries for analysis        |
| log_connections / log_disconnections | 1 (on)                 | Audit for security                   |
| rds.log_retention_period             | 10080 minutes (7 days) | CloudWatch logs retention            |

## Appendix C – Sample Query Patterns with Index Usage

### Find active batch for Gubae Abew:

```
sql
SELECT * FROM lms_batches
WHERE course_track = 'GUBAE_ABEW' AND status = 'ACTIVE';
-- Uses partial index idx_lms_batches_active
```

### Get unread notifications for a user (sorted newest first):

```
sql
SELECT * FROM notifications
WHERE user_id = '...' AND is_read = FALSE
ORDER BY created_at DESC;
-- Uses index (user_id, is_read, created_at)
```

### Monthly audit log retrieval for a specific entity:

```
sql
SELECT * FROM audit_logs
WHERE entity_type = 'user' AND entity_id = '...'
AND created_at BETWEEN '2025-01-01' AND '2025-02-01';
-- Partition pruning + btree index on created_at
```